

# AI-Driven Development with Azure, OpenAI, and GitHub







# AI-Driven Development with Azure, OpenAI, and GitHub



Principal Trainer  
Xebia USA | Microsoft Services



# Thank you to our Sponsors!



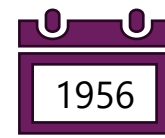
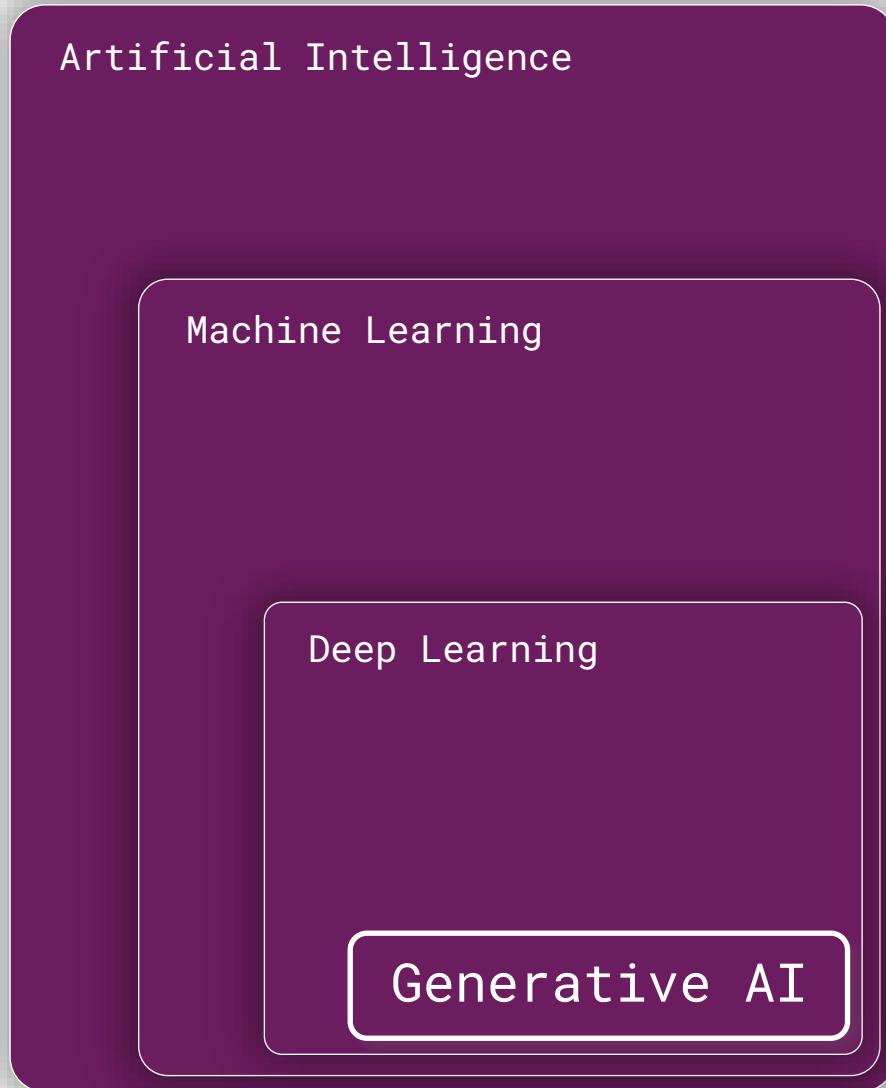


# Value from AI-Driven Development

- **Cuts Costs, Automates, and Improves Quality**
  - Automates search, reduces manual work, and boosts quality with trusted data.
- **Enhances Experience for Customers and Employees**
  - Delivers fast, accurate answers and personalizes support using your real knowledge base.
- **Drives Better Decisions and Faster Innovation**
  - Uses fresh data for smarter choices and faster, more relevant insights.
- **Builds a Competitive, Secure, and Scalable Foundation**
  - Scales with your content and strengthens security by grounding answers in approved sources

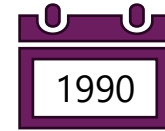


# Brief History of AI



## Artificial Intelligence

The field of computer science that seeks to create intelligent machines that can replicate or exceed human intelligence.



## Internet and Big Data

The rise of the internet and the availability of big data significantly advanced "machine learning" methods.



## Machine Learning

Subset of AI that enables machines to learn from existing data and improve upon that data to make decisions or predictions. (IBM's Deep Blue defeated world chess champion)



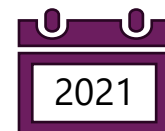
## ImageNet challenge

AlexNet's victory in the ImageNet challenge (image recognition) significantly advanced "deep learning", catalyzing its acceptance..



## Deep Learning

A machine learning technique in which layers of neural networks are used to process data and make decisions.



## Generative AI

Create new written, visual, and auditory content given prompts or existing data.

# Vocabulary Used in AI

## OpenAI

AI research lab advancing and promoting beneficial AI.

## ChatGPT

OpenAI's AI model for generating conversational text.

## Prompt Engineering

Crafting prompts to optimize AI responses.

## Tokens

Basic text units used by AI for language processing.

## Embeddings

Numerical representations of words or entities

## Completions

AI-generated responses to given prompts.

## Semantic Kernel / LangChain

Two different AI development frameworks for building AI apps

## LLM

Large Language Model, model that stores the relation between tokens



# Prompt Engineering





# What is Prompt Engineering?

Prompt engineering is the process of designing and creating **high-quality prompts** that can be used to generate accurate and useful **suggestions** with AI.

“The art of coaxing the best possible result out of a large language model”

Jennifer Marsman  
Principal Engineer, Microsoft AI Research





# Prompt Engineering Techniques

## Zero-shot prompting

- ✓ The model is given only the task description, with no examples.
- ✓ Used to generate answers based on instructions alone.

## One-shot and n-shot prompting

- ✓ Provide one (one-shot) or several (n-shot) example inputs and outputs to set the context.
- ✓ The model uses these examples to produce similar, often more accurate results.

## Chain-of-Thought reasoning

- ✓ Guide the model through step-by-step reasoning to solve more complex tasks.
- ✓ Helps the model break down the problem and reach a better answer.



# Basics of prompt engineering

## Zero-Shot Prompt

Create a c# model for a FlightLogSignature property.

## One-Shot Prompt

Example: 17121903-DEP-ARR-WB001

- 17th of December 1903
- Departure from Kitty Hawk, NC
- Arrival at Manteo, NC
- Flight number WB001

## n-Shot Prompt

Technical Requirements

- Create a FlightLog record type
- Add a Parse method to the FlightLog record type
- The Date must be a DateTime.







# Prompting Strategy

## Ensure Clarity

Provide context, write clearly, and give Copilot quality input for better results

## Group and Itemize Tasks

Breaking down complex tasks and use previous outcomes for efficiency

## Organized approach

Leverage prompting patterns to solve complex problems



[www.promptingguide.ai](http://www.promptingguide.ai)



# AI for Coding – What Works Best!

## AI speeds you up, but you're still in charge

Treat AI like a teammate—guide it, review its work, and refine results. You are the pilot!!!

## Security comes first

Trust but verify. Always check and secure AI-generated code.

## Your expertise matters

The clearer your vision and knowledge, the better the AI helps you.

## Iterate and learn

Coding with AI is about experimenting and improving together.

*AI can boost productivity, but real progress comes from learning, adapting, and combining your skills with AI.*



# Retrieval Augmented Generation





# Why a RAG app?

What an LLM “knows”



# Why a RAG app?

What an LLM “knows”



Not on the public internet  
or  
Recently updated



RAG expands the knowledge of large language models (LLMs) from their initial training data to external datasets you provide.



# Value from Retrieval Augmented Generation Apps

- Gives **accurate answers** using your latest documents and data
- Reduces errors by **grounding responses** in real sources
- Makes knowledge easy to **search** with **natural language**
- Powers **smarter support, research,** and **compliance** with **your content**





# Use Cases for RAG Applications



**Accurate answers from patient records and medical guidelines**  
Saves time for care teams and reduces diagnostic errors



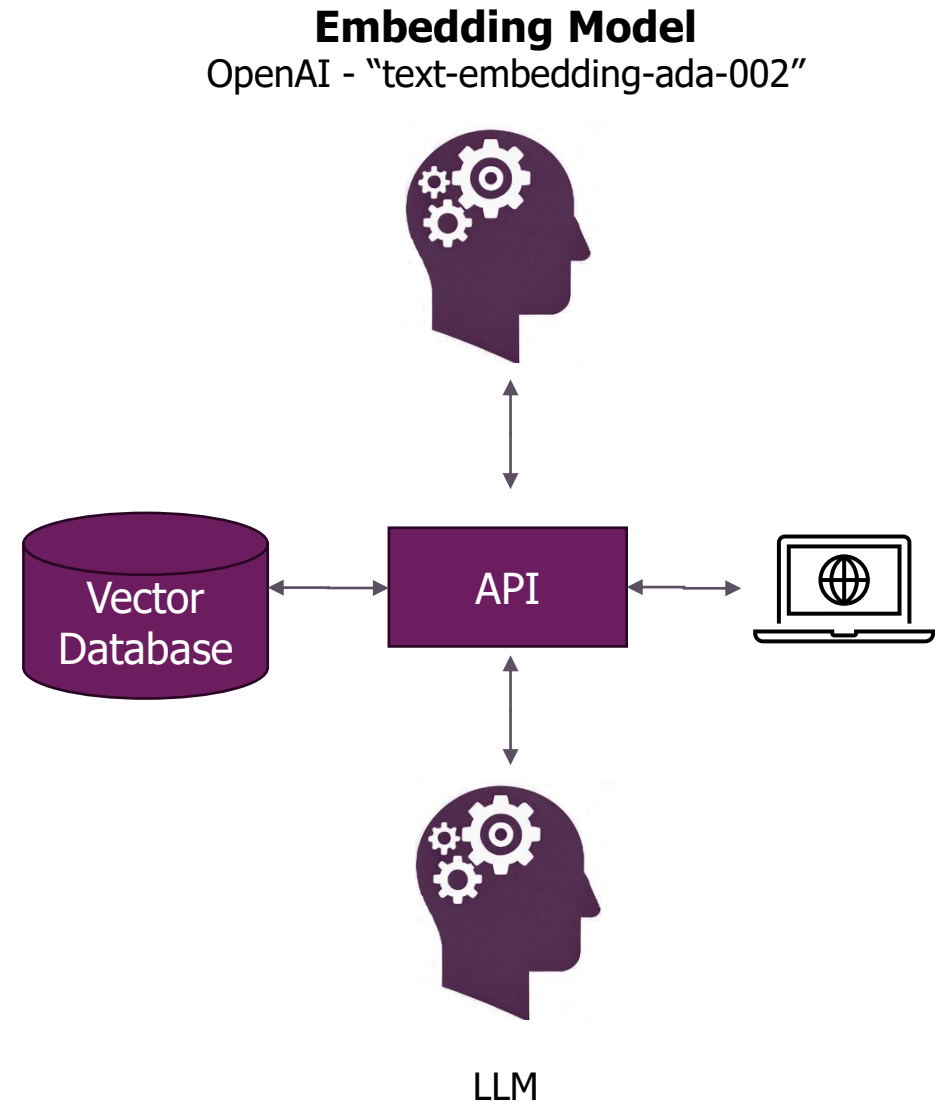
**Guided troubleshooting with manufacturer data and diagnostics**  
Prevents costly mistakes and improves technician efficiency



**Up-to-date compliance answers from regulatory documents**  
Lowers compliance risk and speeds up audit readiness

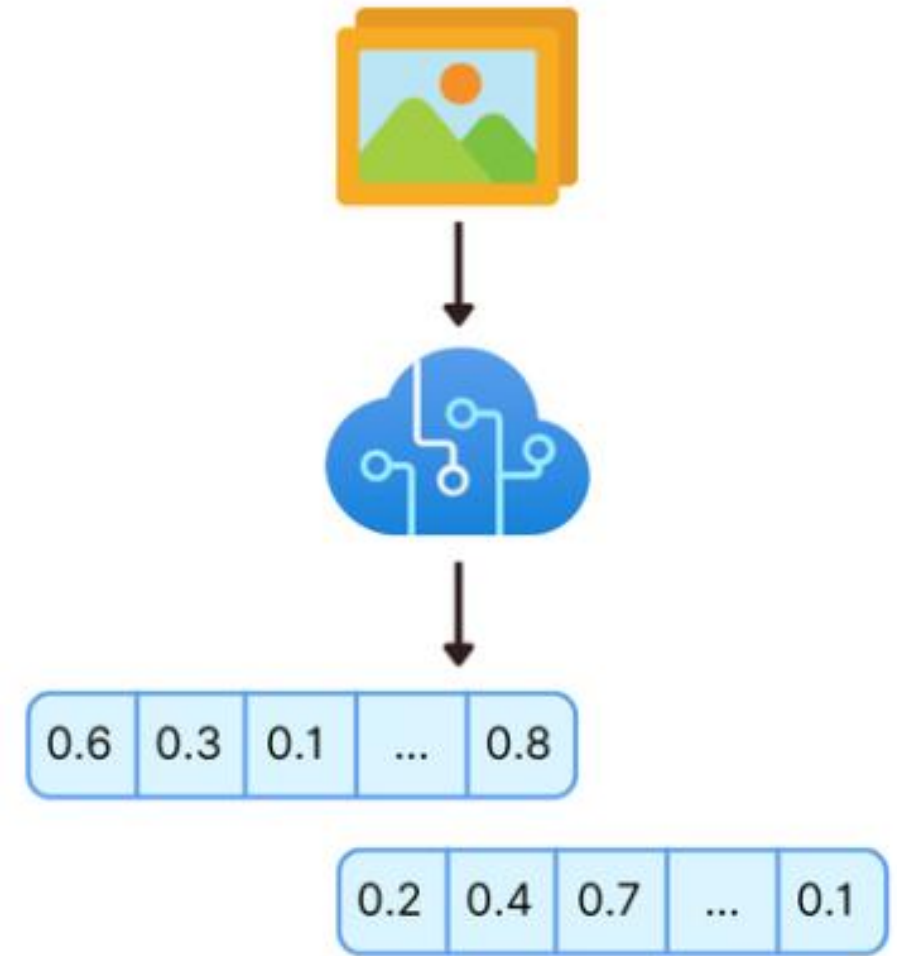
# What is a RAG app?

- ✓ Vector databases contain “**embeddings**” which are numerical representations of data processed by an embedding model
- ✓ A RAG app typically uses an API or orchestrates requests made from a client.
- ✓ The API uses the request from the client to get an embedding from the embedding model.
- ✓ The embedding from the embedding model is sent to vector database, which performs a similarity search.
- ✓ The results from the similarity search are enriched by a prompt and sent to an LLM for analysis. The LLM returns results, which are sent to the client.



# What are Embeddings?

- ✓ Numerical representation of text (**vectors**) that captures semantics and relationships between words.
- ✓ Embedding models capture features and nuances of the text.
- ✓ Rich embeddings can be used to compare text similarity.
- ✓ Multilingual Text Embeddings can identify meaning in different languages.



“I heard a dog bark loudly at a cat” >>> 40,6755,264,5679,54842,54945,520,264,8415



# What is LangChain?

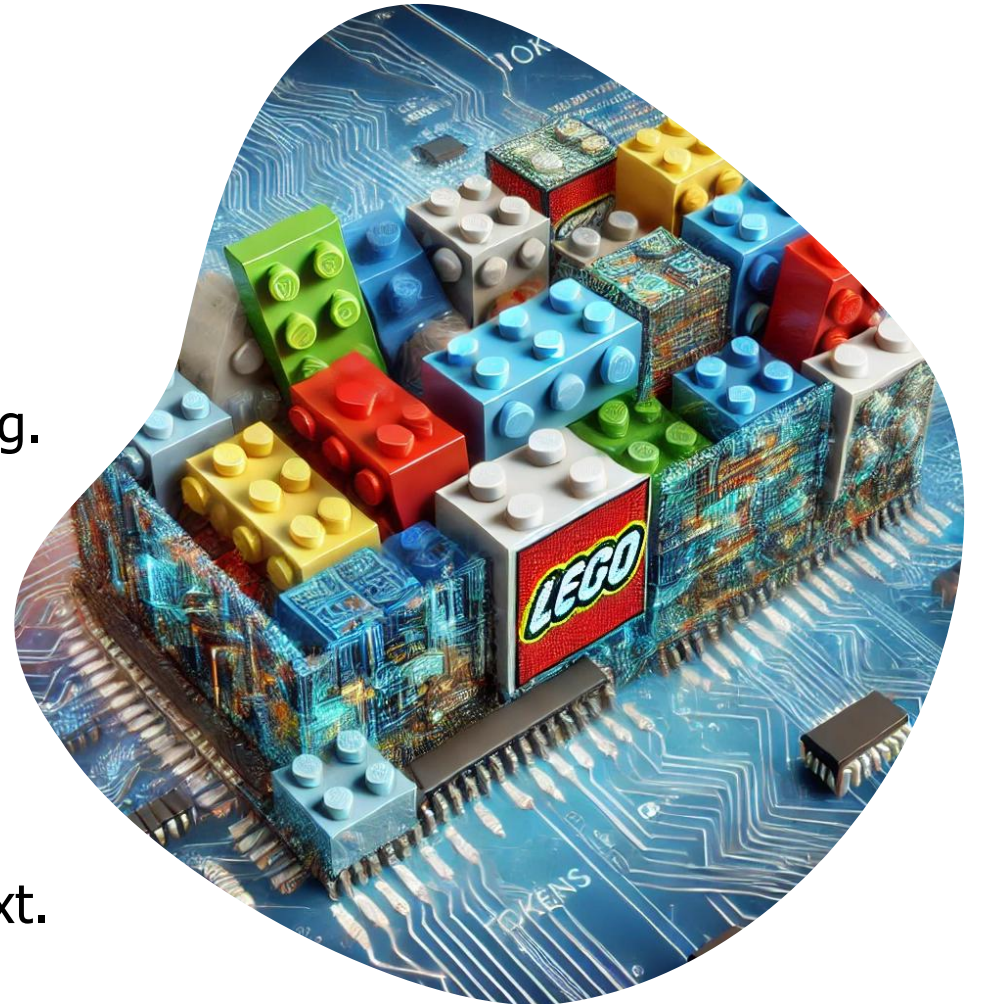
- ✓ LangChain is a framework that helps developers build applications using language models like GPT-4.1, Claude, or Mistral.
- ✓ It simplifies the creation of workflows where these models can answer questions, generate content, or trigger actions.
- ✓ LangChain enables integration of language models into real-world tools and services, helping teams solve problems more efficiently and intelligently.



# Building Language with Tokens

"**Tokens** are the **language Lego pieces** that models use to build and understand human language."

- Tokens are the fundamental units for language processing.
- Each token is converted into a numerical format that represents its meaning.
- The model uses these numbers to analyze and generate responses.
- Final output is transformed back into human-readable text.



# Context Lengths and Tokens

```
using eShop.Catalog.API.Services;

public static class Extensions
{
    public static void AddApplicationServices(this IHostApplicationBuilder builder)
    {
        builder.AddNpgsqlDbContext<CatalogContext>("catalogdb", configureDbContextOptions: dbContextOptionsBuilder =>
        {
            dbContextOptionsBuilder.UseNpgsql(builder =>
            {
                builder.UseVector();
            });
        });

        builder.Services.AddMigration<CatalogContext, CatalogContextSeed>();

        builder.Services.AddTransient<IIntegrationEventLogService, IntegrationEventLogService<CatalogContext>>();
    }
}
```



# Context Lengths and Tokens

```
using eShop.Catalog.API.Services;

public static class Extensions
{
    public static void AddApplicationServices(this IHostApplicationBuilder builder)
    {
        builder.AddNpgsqlDbContext<CatalogContext>("catalogdb", configureDbContextOptions: dbContextOptionsBuilder =>
        {
            dbContextOptionsBuilder.UseNpgsql(builder =>
            {
                builder.UseVector();
            });
        });

        builder.Services.AddMigration<CatalogContext, CatalogContextSeed>();

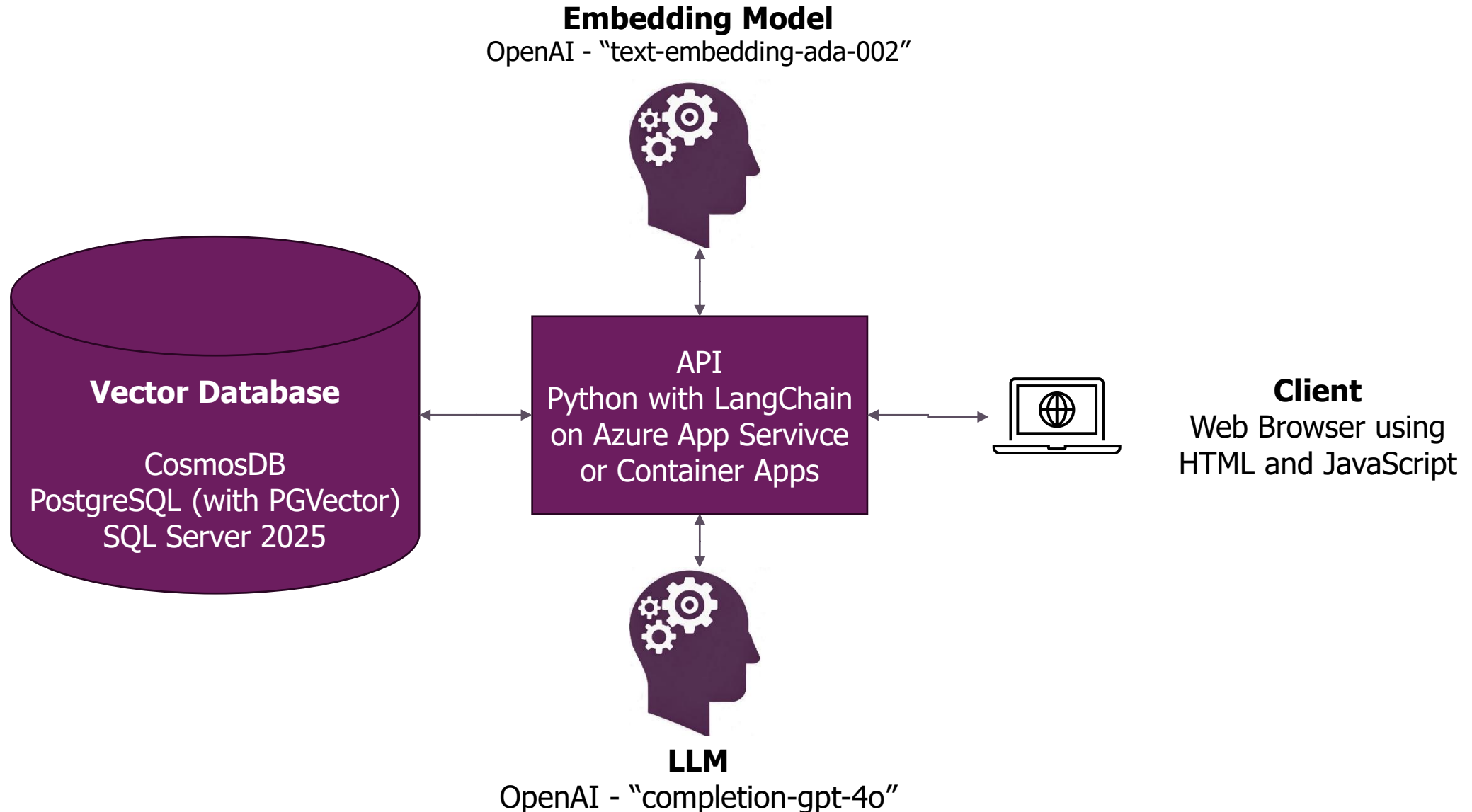
        builder.Services.AddTransient<IIntegrationEventLogService, IntegrationEventLogService<CatalogContext>>();
    }
}
```

# Context Lengths and Tokens

What  
Happens If  
your  
Prompt +  
Context  
Length is  
too Long?



# Azure components for building RAG apps





# JEOPARDY!

## Example: Jeopardy Dataset (55.55 MiB)

JSON file contains **216,930 Jeopardy questions, answers and other data.**

The file takes **10+ hrs to load** and is an unordered list of questions as follows:

- ✓ **'category'** : the question category, e.g. "HISTORY"
- ✓ **'value'** : \$ value of the question as string, e.g. "\$200"
- ✓ **'question'** : text of question
- ✓ **'answer'** : text of answer
- ✓ **'round'** : one of "Jeopardy!", "Double Jeopardy!", "Final Jeopardy!" or "Tiebreaker"
- ✓ **'show\_number'** : string of show number, e.g. '4680'
- ✓ **'air\_date'** : the show air date in format YYYY-MM-DD

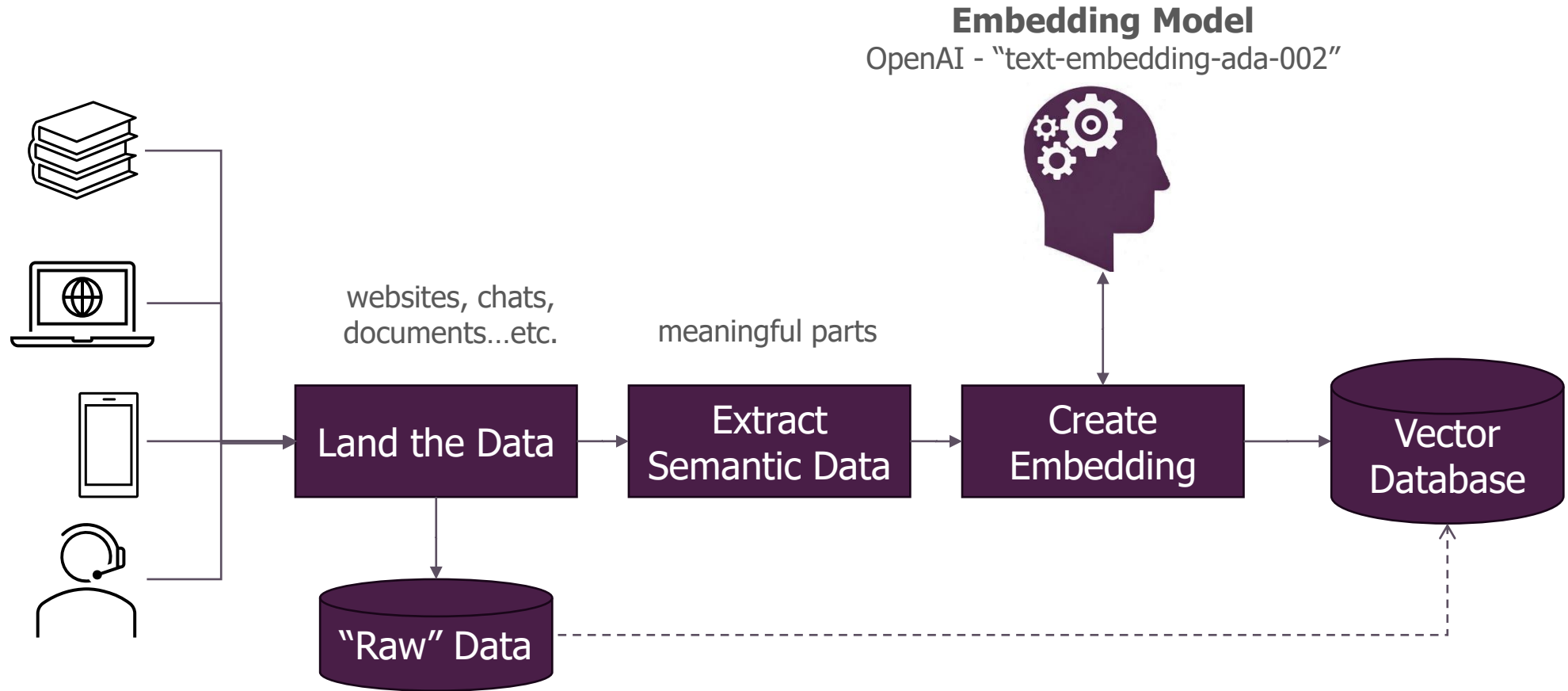
# Example: Jeopardy Dataset (55.55 MiB)

json file contains **216,930** Jeopardy questions, answers and other data.

The file takes **10+ hrs to load** and is an unordered list of questions as follows:

- **'category'** : the question category, e.g. "HISTORY"
- **'value'** : \$ value of the question as string, e.g. "\$200"
- **'question'** : text of question
- **'answer'** : text of answer
- **'round'** : one of "Jeopardy!", "Double Jeopardy!", "Final Jeopardy!" or "Tiebreaker"
- **'show\_number'** : string of show number, e.g '4680'
- **'air\_date'** : the show air date in format YYYY-MM-DD

# Loading the database! (RAG Pre-processing)



This entire pre-processing step is offline. It's done once, ahead of time, so your RAG app can respond in real-time later—kind of like indexing a book before looking something up.



# Demo

## (setup + loading)

Set up a database-backed RAG app

- ✓ **building the Azure Resources**
- ✓ **splitting a text file** into chunks
- ✓ **generating embeddings** for each chunk
- ✓ store in a SQL Server **vector** database as **embedding**



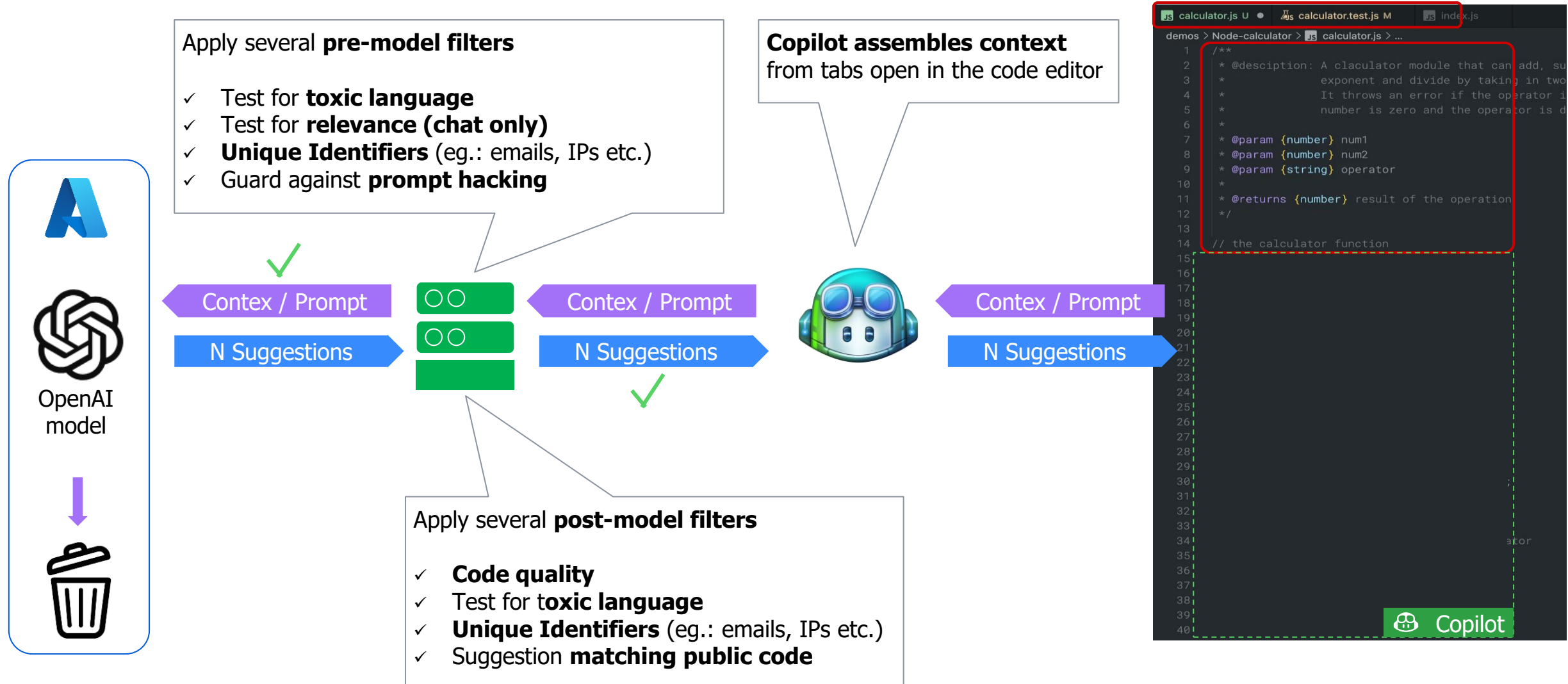


# GitHub Copilot

- ✓ AI-driven insights and suggestions
- ✓ Pair programming, real-time collaboration
- ✓ Context comments & Code
- ✓ Individual Lines and whole functions
- ✓ Integrates with popular IDEs
- ✓ Model jointly developed by GitHub, Microsoft, and OpenAI.



# How does GitHub Copilot work?





# GitHub Chat Mode Differences

Next evolution of AI-assisted development



## Copilot Ask

natural, conversational assistance



## Copilot Edits

update with clear intent



## Copilot Agent

step-by-step task guidance

|               | Ask mode                                                | Edits mode                                        | Agent mode                                            |
|---------------|---------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------|
| Best Use Case | Quick help, explanations, code suggestions              | Refactoring, code fixes, rewriting blocks of code | Multi-step workflows, tasks across files/projects     |
| Strengths     | Fast, easy to use, great for Q&A and inline completions | Efficient for modifying existing code             | Handles goals, file edits, planning, and coordination |
| Limitations   | Context can be limited to current file/scope            | Less flexible for multi-step or logic-heavy tasks | Still evolving, slower, may require cleanup           |



# Understanding Premium Requests

## What are they?

**Premium requests** happens when you use advanced Copilot features or powerful AI models.

## When do I use them?

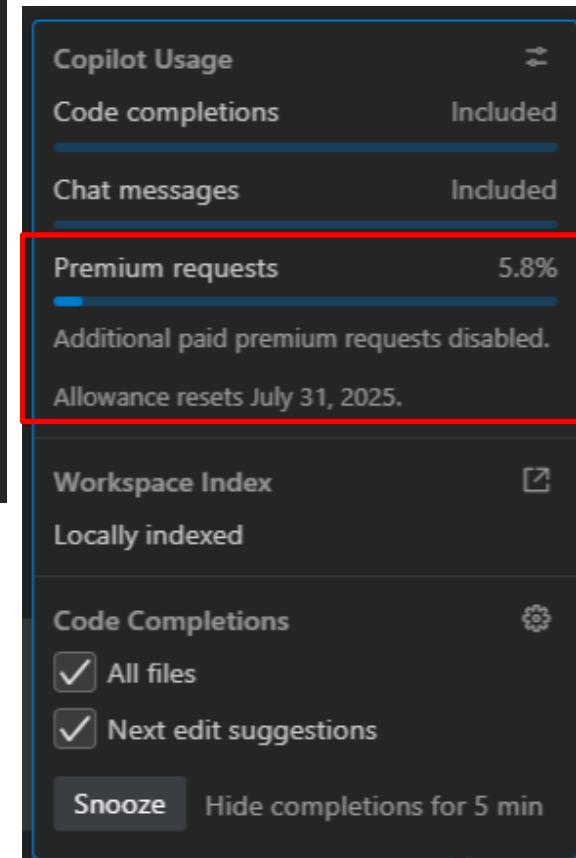
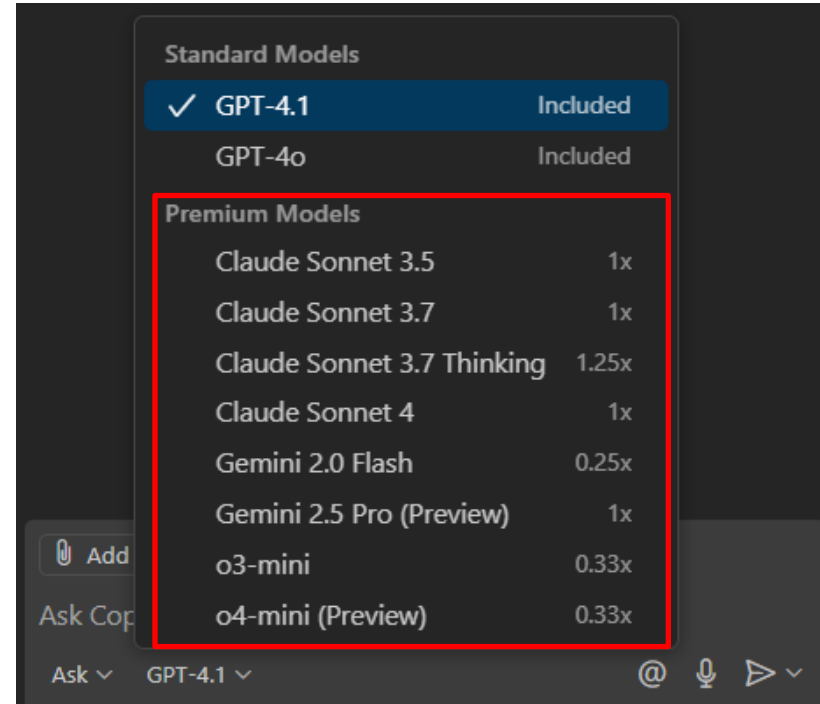
- ✓ Use **Copilot Ask, Edit, or Agent Mode**
- ✓ Choose **higher-end AI models**
- ✓ Interact via **Copilot Extensions**

## Why does it matter?

- ✓ Each developer has a monthly limit
- ✓ Overages may cost **\$0.04 per request**
- ✓ Base model = free for paid users
- ✓ Manage your usage wisely to avoid unneeded costs

## Best Practices

- ✓ Stick to the base model unless advanced help is needed
- ✓ Track your usage in GitHub → Settings → Copilot



# GitHub Copilot Coding Agent

Next evolution of AI-assisted development

## What It Does

- ✓ Creates new files and updates existing ones
- ✓ Breaks down large features into manageable pieces
- ✓ Orchestrates multiple code and config changes across files
- ✓ Provides guidance through complex, multi-stage development tasks

## When to Use

- ✓ Starting new features that touch multiple files
- ✓ Implementing workflows spanning APIs, tests, and application logic
- ✓ Managing long-running tasks where the agent can iterate and improve



|                     | Best Use Case                                                               | Strengths                                                                                                   | Limitations                                                                                                |
|---------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Coding Agent</b> | Multi-step tasks that require design, file creation, and long-running logic | Handles complex workflows: sets goals, plans the solution, and manages file edits across the entire project | Requires more setup; can take longer to complete and may produce intermediate drafts that need refinement. |

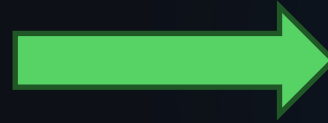


# Large Language Model (LLM)



# How does an LLM work?

"I heard a dog bark loudly at a cat"



|                                   |         |
|-----------------------------------|---------|
| I                                 | (40)    |
| heard                             | (6755)  |
| a                                 | (264)   |
| dog                               | (5679)  |
| bark                              | (54842) |
| loudly                            | (54945) |
| at                                | (520)   |
| ("a" is already tokenized as 264) |         |
| cat                               | (8415)  |



Tokenized Array for ingestion by the LLM:

[ 40, 6755, 264, 5679, 54842, 54945, 520, 264, 8415 ]



# How does an LLM work?

"I heard a dog bark loudly at a cat"



Tokenized

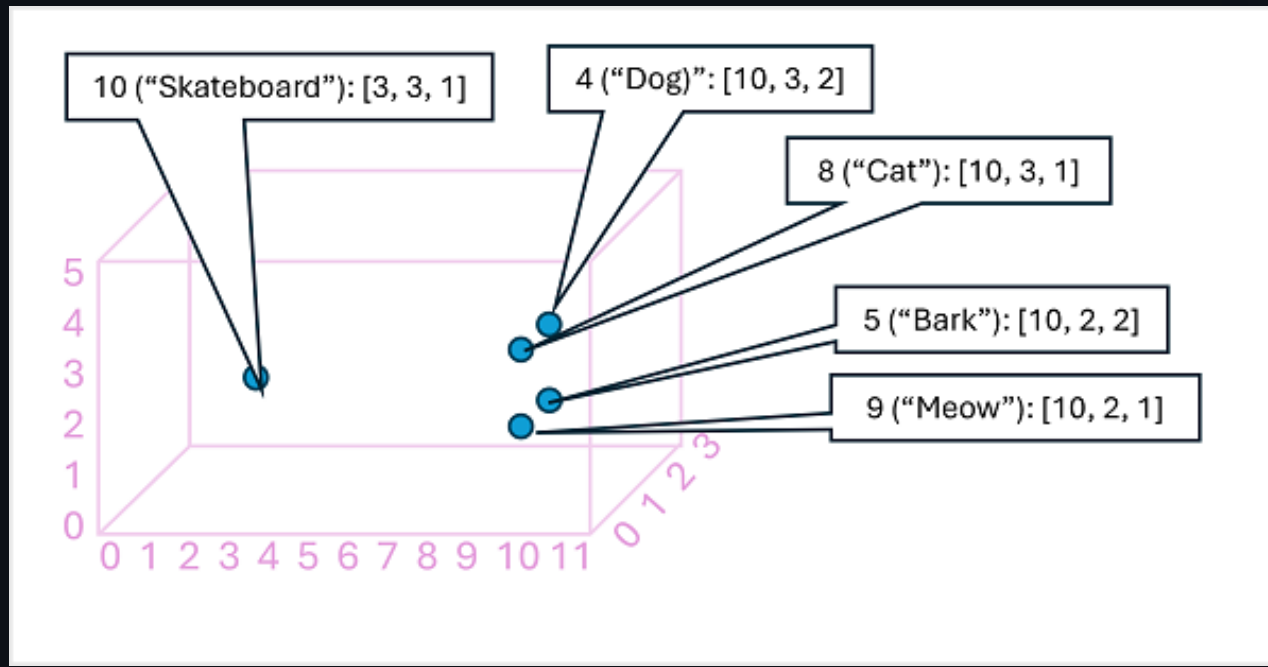
[ 40, 6755, 264, 5679, 54842, 54945, 520, 264, 8415 ]

"I heard a cat"



Tokenized

[40, 6755, 264, 8415]



Based on the **tokens** and **vectors**, propose the **next word** with **high probability**



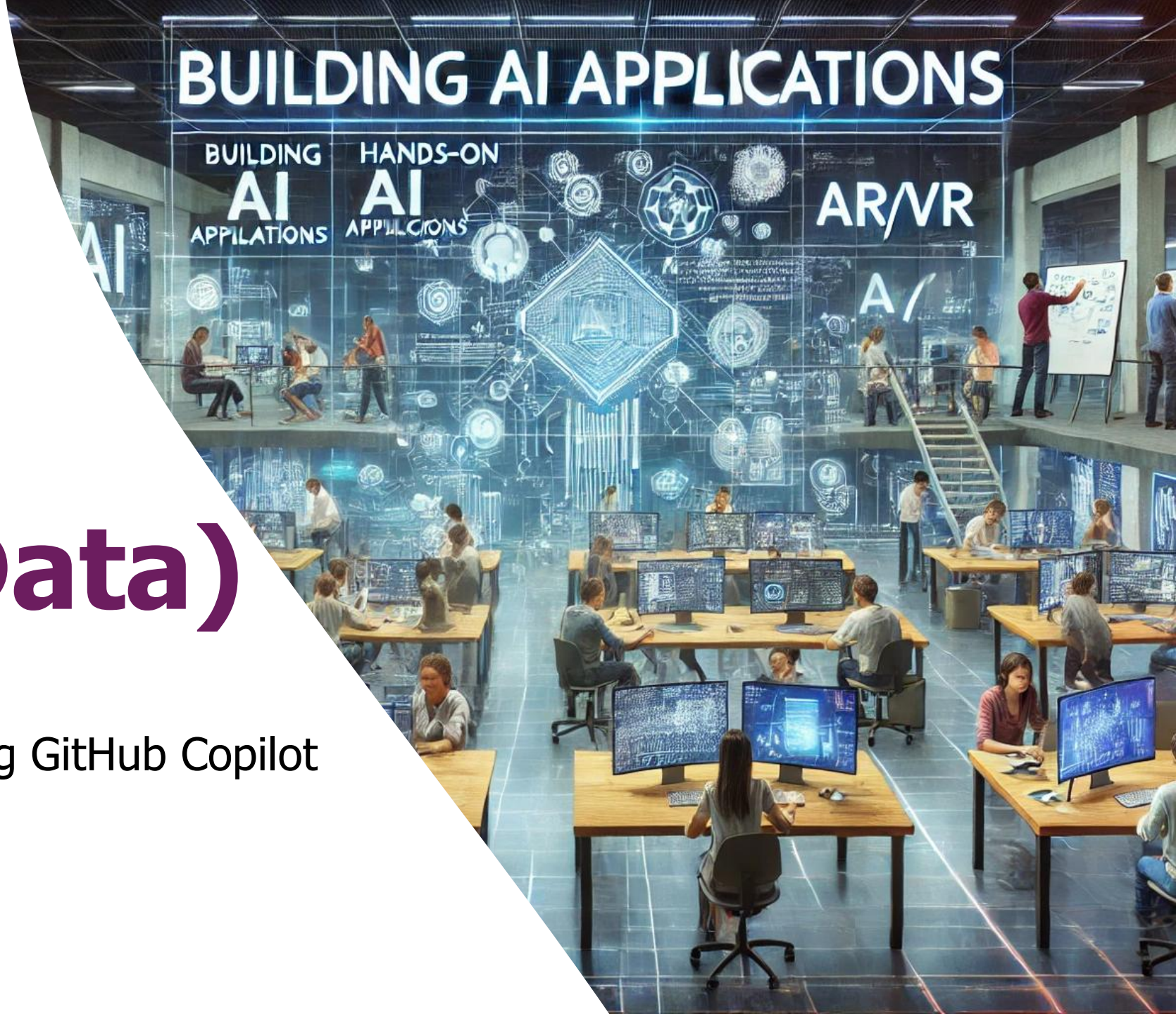
"I heard a cat **meow** at the **dog**"



# Demo

## (Zero to Data)

- ✓ Prompt Engineering using GitHub Copilot
- ✓ API Integration Testing





# Front End Design

- ✓ HTML
- ✓ React
- ✓ Vue.js
- ✓ Angular
- ✓ Svelte
- ✓ Vanilla JavaScript





# Build Intuitive HTML Frontends



Modern GUI frontend for making API calls to Flask backend.

Popular frameworks and libraries.

- ✓ **React.** A JavaScript library for building user interfaces.
- ✓ **Vue.js.** A progressive JavaScript framework for building user interfaces.
- ✓ **Angular.** A platform for building mobile and desktop web applications.
- ✓ **Svelte.** A compiler that generates minimal and highly optimized JavaScript code.
- ✓ **Vanilla JavaScript.** Using plain HTML, CSS, and JavaScript without any frameworks.



# Vanilla JavaScript



## CSS Styling

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Jeopardy Question Search</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 50px;
      text-align: center;
    }
    input[type="text"], textarea {
      width: 80%;
      padding: 10px;
      margin: 10px 0;
    }
    button {
      padding: 10px 20px;
      font-size: 16px;
      cursor: pointer;
    }
    #response {
      margin-top: 20px;
      white-space: pre-wrap;
    }
  </style>
</head>
```

The “**Form**” uses the GET method to submit the search query.  
A text “**Input**” for the user to enter their search query.  
A submit “**Button**” to send the form data.

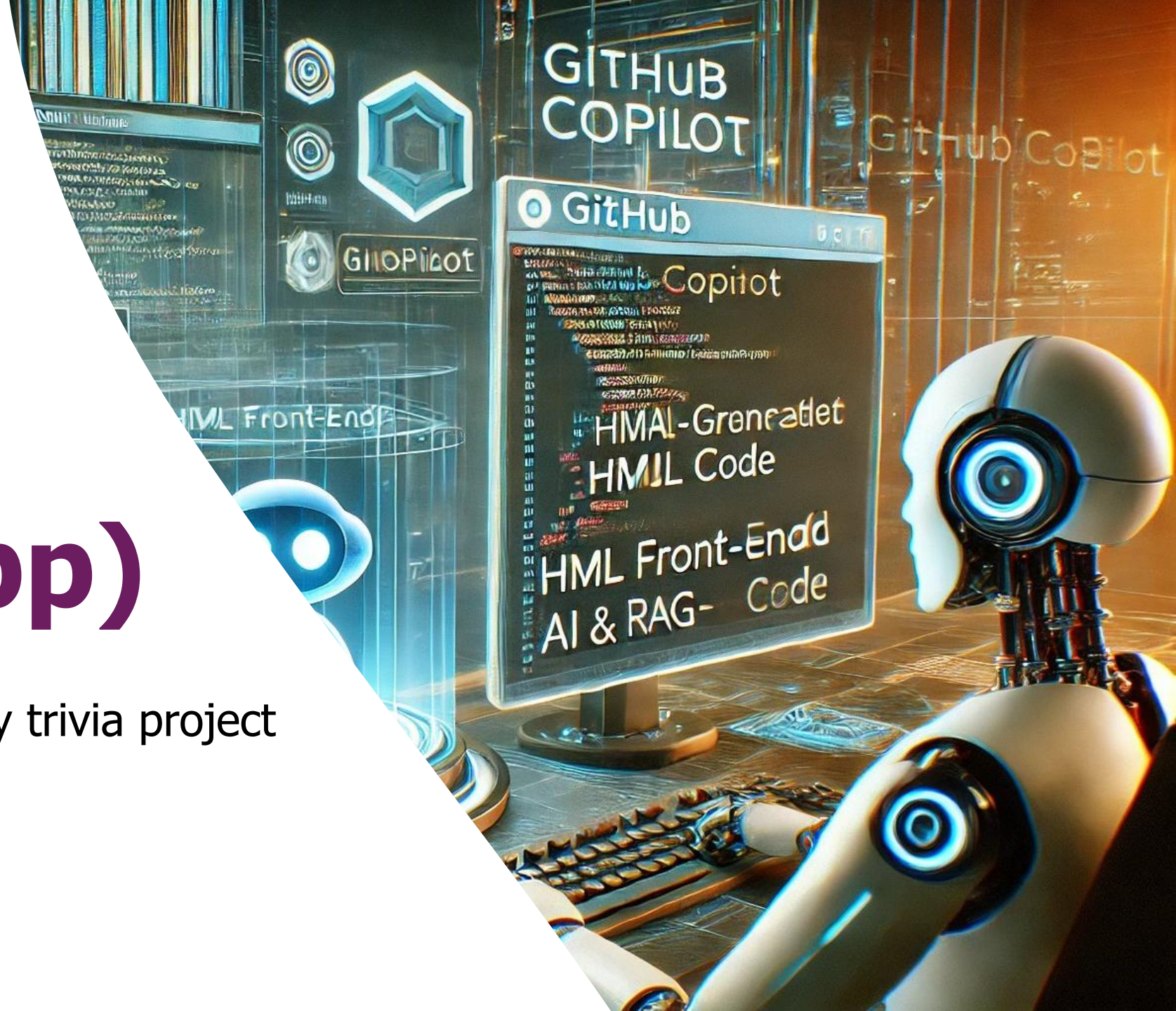
A “**Div**” to display the search query.

```
<body>
  <h1>Jeopardy Question Search</h1>
  <form method="GET">
    <input type="text" name="query" placeholder="Enter your search query">
    <button type="submit">Search</button>
  </form>
  <div id="response">
    <!-- Assume this is where the user input is echoed back -->
    <script>
      const params = new URLSearchParams(window.location.search);
      const query = params.get('query');
      if (query) {
        document.getElementById('response').innerText = query;
      }
    </script>
  </div>
</body>
</html>
```

# Demo

## (API to App)

✓ Frontend to the Jeopardy trivia project





# Copilot Autofix

**Securing the World's  
Software with GitHub  
Advanced Security**





# Why GitHub Advanced Security?

- ✓ **Traditional AppSec = too little, too late**
  - ✓ Security becomes a backlog item
- ✓ **80–90% of app code is open source**
  - ✓ Unmonitored dependencies = major risk
- ✓ **Developer mistakes (exposed tokens, unsafe code)**
  - ✓ cause real breaches
- ✓ **570x more developers than security researchers**
  - ✓ We need scale
- ✓ **GHAS + GitHub Copilot = scalable**
  - ✓ Proactive protection built into your workflow

# GitHub Advanced Security



## Code Security

Code scanning analyzes code for vulnerabilities using CodeQL and provides alerts in pull requests or on push



## Dependency scanning

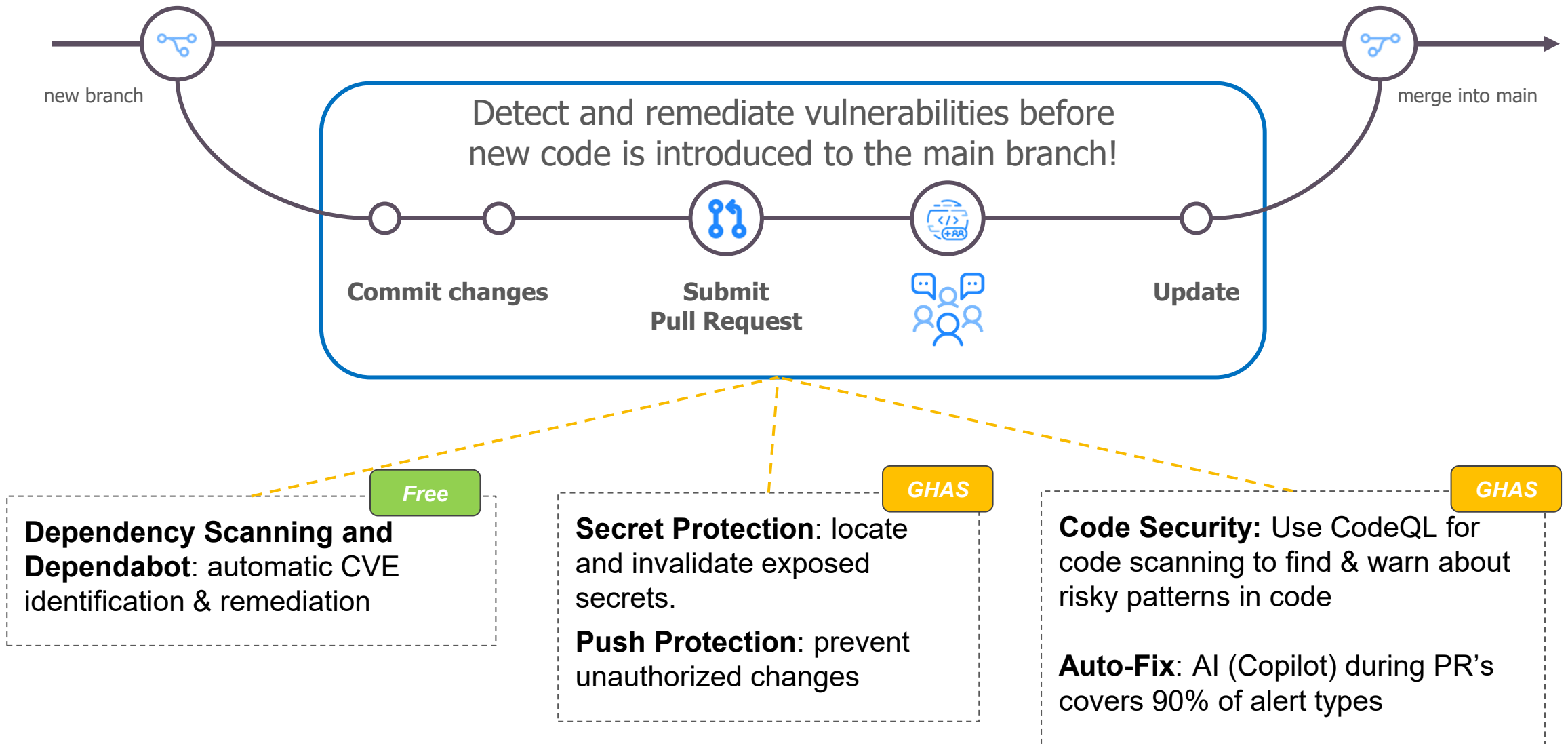
Automatically detects known vulnerabilities in existing dependencies creating Dependabot alerts.



## Secret Protection

Detects hardcoded secrets in your codebase, both in commit history and before they're pushed.

# Developer First Tools



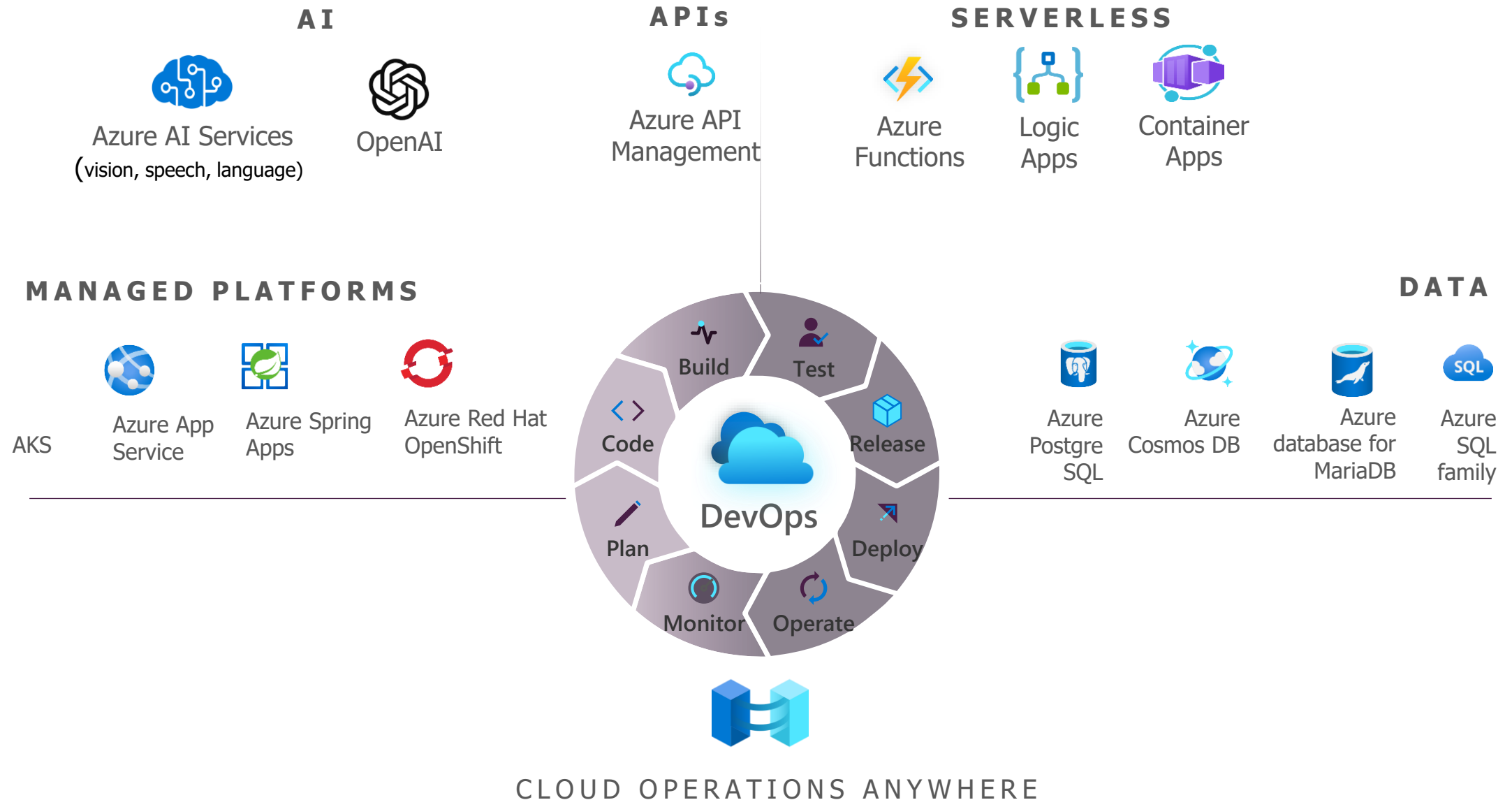


# Demo

✓ **GitHub Advanced Security**



# Modernizing Apps on Azure



# Appendix

Azure OpenAI Service

[Azure OpenAI Service Overview](#)

[Azure OpenAI Service Documentation](#)

How a LLM Works

[Understanding Large Language Models](#)

Prompt Engineering & AI

[Prompt Engineering Guide](#) | [Prompting for GPT-3](#) | [LinkedIn - Learning by building: AI Assisted Development](#)

RAG Architecture

[Retrieval-Augmented Generation \(RAG\) Architecture Explained](#)

Building RAG Apps with LangFlow

[LangFlow Documentation](#) | [LangFlow GitHub Repository](#)

GitHub Advanced Security

[GitHub Advanced Security](#) | [Introduction to GitHub Advanced Security](#)

GitHub Copilot

[GitHub Copilot Documentation](#)

Deploying to Microsoft Azure

[Azure Deployment Guide](#)



Slides

<https://github.com/PagelsR/Talks>

# Thank you!



**Randy Pagels**

Principal Trainer

Xebia USA | Microsoft Services

